

LEGRAND Romain

BERNARD Yeleen

Rapport de projet : planning poker

1. Introduction et contexte

L'objectif du projet est de créer une application de planning poker, utilisable sur plusieurs machines en même temps en LAN. L'application permet de créer une réunion, de s'y connecter, et de voter pour chaque user story selon le mode de vote choisi lors de la création de la réunion. Les users stories sont contenues dans un fichier JSON, qui peut être importé, créé ou modifié depuis l'application elle-même. L'application dispose également d'un chronomètre réglable lors de la création de la réunion, qui impose un temps maximum pour voter.

Nous avons travaillé sur ce projet en binôme. Nous avons commencé par faire l'inventaire des différentes tâches, estimer leur difficulté, et nous les répartir en fonction de nos compétences. L'objectif était donc de se baser sur nos forces dans certains domaines d'un côté, et d'apprendre de nouvelles choses de l'autre, notamment grâce au pair programming que nous avons essayé de mettre en place. Nous avons commencé par l'intégration continue, avec un workflow qui build automatiquement le projet à chaque push GitHub, ce qui nous a permis d'identifier des erreurs rapidement. En parallèle, nous avons commencé le développement du projet avec un planning poker « pour une personne », avec la création d'interfaces et la gestion des fichiers JSon. Notre planning a ensuite été retardé à cause de problèmes de santé de Yeleen, et Romain a commencé à travailler sur le projet de son côté, mais nous n'avons pas pu nous concerter sur certains points de décision, par exemple la nomenclature ou les conventions de codage.

Nous avons essayé de mettre en place des commits réguliers avec une review du code de chacun dès que possible, pour que nous puissions tous les deux savoir ce que faisait l'autre et le fonctionnement exact des features mises en place. Étant donné le fonctionnement de Unity, nous étions obligés de souvent utiliser le code écrit par l'autre, et cela nous a également poussé à toujours bien commenter l'ensemble de notre code et à communiquer. Nous avons toutefois travaillé sur des branches différentes autant que possible pour éviter des problèmes de merge, en sachant que chaque feature a été construite sur une branche à part, avant d'être merge sur la branche Develop. À la fin, la branche Develop rassemblait donc l'intégralité de notre code, nous avons corrigé les dernières erreurs dues au merge des différentes features, et nous l'avons merge avec la branche main.

2. Choix techniques et architecture

Nous avons choisi de travailler sur ce projet avec le moteur 3D Unity, qui permet de créer des jeux vidéo et des applications. L'ensemble du code, à l'exception des fichiers d'intégration continue, est donc écrit en C#. Nous avons fait ce choix car le C# est un langage que nous maîtrisons bien tous les deux, et surtout parce qu'Unity permet de créer des interfaces plus poussées, en utilisant par exemple des modèles 3D, que nous avons fait.

Ensuite, différents frameworks et packages se sont ajoutés au projet au fur et à mesure de nos besoins.

Pour les fichiers JSON, nous avons utilisé un package téléchargé sur le package manager d'Unity, Standalone File Browser, qui permet d'ouvrir une fenêtre pour parcourir les fichiers de la machine, ce que Unity ne permet pas nativement. Ce package est donc utilisé à deux reprises, pour charger un fichier JSON déjà présent dans la machine ou enregistrer un fichier JSON créé via l'application à l'endroit désiré.

Pour tester notre système de multijoueur, nous avons utilisé le package ParrelSync, téléchargé également via le package manager par url git. Il permet de lancer plusieurs instances en même temps du projet Unity et donc de tester nos fonctionnalités multijoueur. Ce package est utile uniquement pour les tests, et n'est pas nécessaire pour le fonctionnement du planning poker tel qu'il est pensé pour le client, sur plusieurs machines différentes avec une instance par machine.

Enfin, nous utilisons le framework PurrNet qui facilite la mise en place d'un système multijoueur. Il rend abstrait les couches de communication, ce qui nous a permis d'implémenter un système multijoueur fonctionnel sans avoir à gérer directement les connexions réseau ou les sockets. Il fournit également des attributs et classes propres au fonctionnement du multijoueur pour en faciliter l'implémentation.

Pour ce qui est de l'architecture du projet, nous utilisons une architecture MVC (Modèle-Vue-Contrôleur), qui est souvent la plus adaptée sur Unity. Le modèle est constitué des classes USData et USJsonFile, qui sont les classes qui contiennent les informations du deck de la réunion (titre, description et score de chaque user story notamment), et gèrent la sauvegarde d'informations dans des fichiers JSON. La vue correspond aux différents éléments d'interface répartis sur plusieurs scènes différentes (boutons, champs de texte, affichage sur un tableau...). Les interactions entre les éléments d'interface et les scripts sont régies par le contrôleur, ici un GameModeManager présent sur la scène qui porte le script GameManager. Cette architecture permet de séparer la redondance d'inclusion, de référencement et de code, et de regrouper tous les fonctionnements liés à l'utilisateur au même endroit. La maintenance et les extensions du code sont également facilitées, par exemple avec de nouveaux modes de vote implémentés si besoin. Un Singleton est également utilisé pour conserver les données

globales entre les différentes scènes de l'application. Il stocke les paramètres de la session (mode de jeu choisi, nombre de participants ainsi que leurs pseudos, durée du chronomètre, réévaluation des users stories déjà notées ou non) ainsi que le deck de users stories. Cela signifie que nous n'utilisons pas de classe Joueur à proprement parler : les pseudos des joueurs sont enregistrés dans le Singleton, et leurs notes sont transmises au GameManager par des instances de Selection (une par joueur).

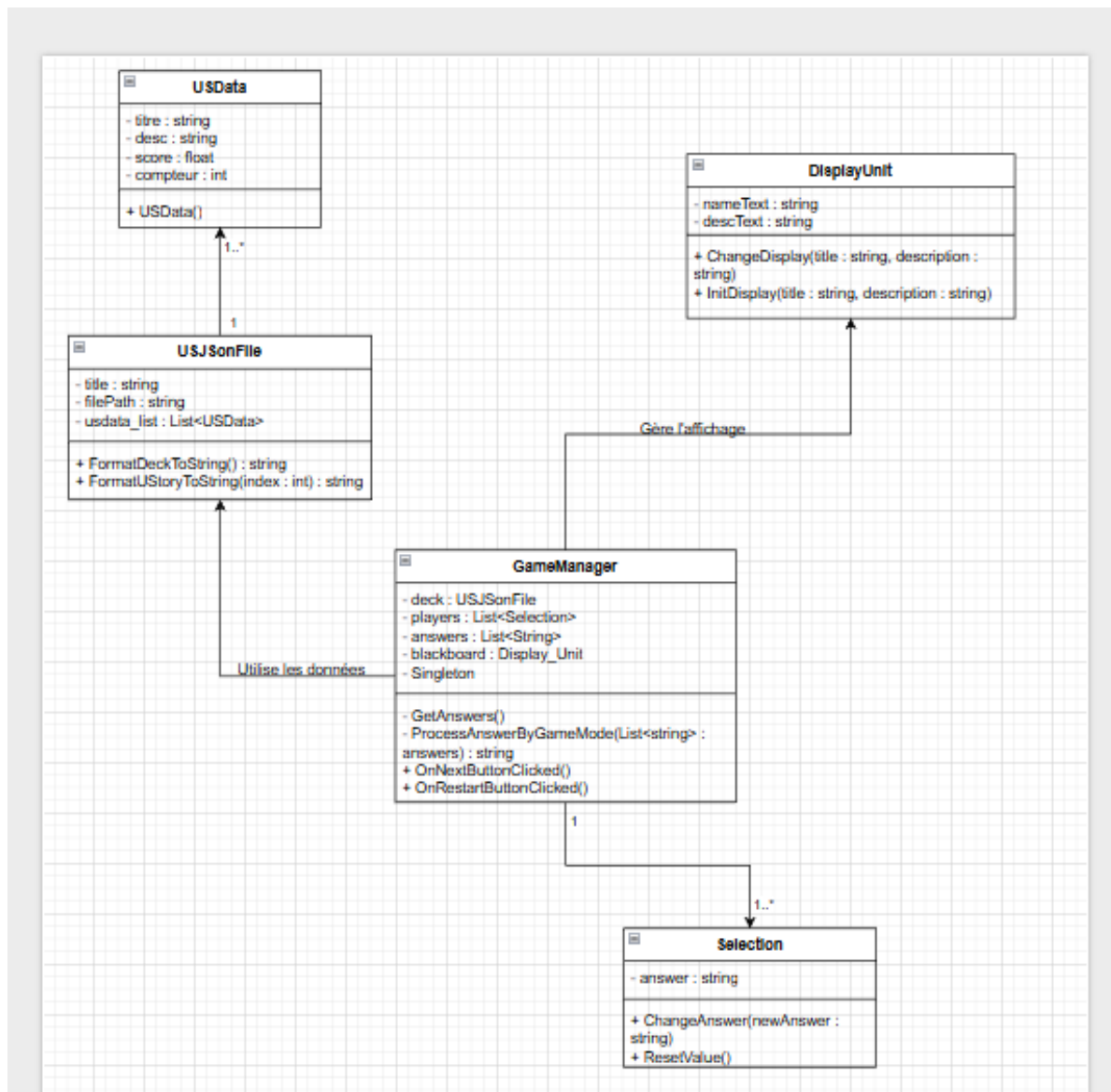


Diagramme de fonctionnement général de notre application de planning poker

Les données de l'application (users stories à estimer et leurs notes) sont enregistrées dans des fichiers JSON, sous forme d'instances de la classe USJsonFile, qui contient un titre et une liste de USData. Les USData sont donc les users stories elle-même, composées d'un titre, d'une description, de la note attribuée et d'un compteur qui permet de suivre le nombre de votes

qui ont été faits pour cette estimation (nécessaire selon le mode de jeu). Les fichiers JSON peuvent être chargés, édités, modifiés et sauvegardés directement depuis l'application. Un système de sauvegarde automatique régulière permet également de ne pas perdre ou corrompre le fichier en cas de problème. La sérialisation et désérialisation sont gérées par la bibliothèque Newtonsoft.Json, implémentée par défaut sur Unity.

3. Mise en place de l'intégration continue

Nous avons utilisé GitHub pour ce projet, et nous nous sommes donc servis de GitHub Actions pour mettre en place l'intégration continue, avec l'automatisation de tests et la génération de documentation automatiques à chaque push ou pull request. Nous avons donc divisé le pipeline en deux workflows, un pour l'automatisation des tests et un pour la génération de documentation, dès le début du projet, ce qui nous a permis de vérifier depuis le départ si notre projet était bien build à chaque push, et donc potentiellement d'éviter des erreurs.

Le fichier test.yaml commence par exécuter les différents tests écrits. Ces tests sont écrits dans un dossier Tests séparé des autres scripts, et utilisent NUnit, la bibliothèque Unity qui permet d'effectuer des tests unitaires notamment. Ils sont divisés en deux catégories : les tests *EditMode*, principalement des tests de fonctions et de classes (tests unitaires) et les tests *PlayMode*, qui simulent le lancement du projet pour vérifier son fonctionnement en se rapprochant des interactions que pourrait avoir un utilisateur. Les tests *PlayMode* servent principalement à vérifier le bon déroulement de l'application, notamment du point de vue de l'affichage. Enfin, le workflow construit un build du projet pour vérifier son fonctionnement.

Les tests écrits sont les suivants :

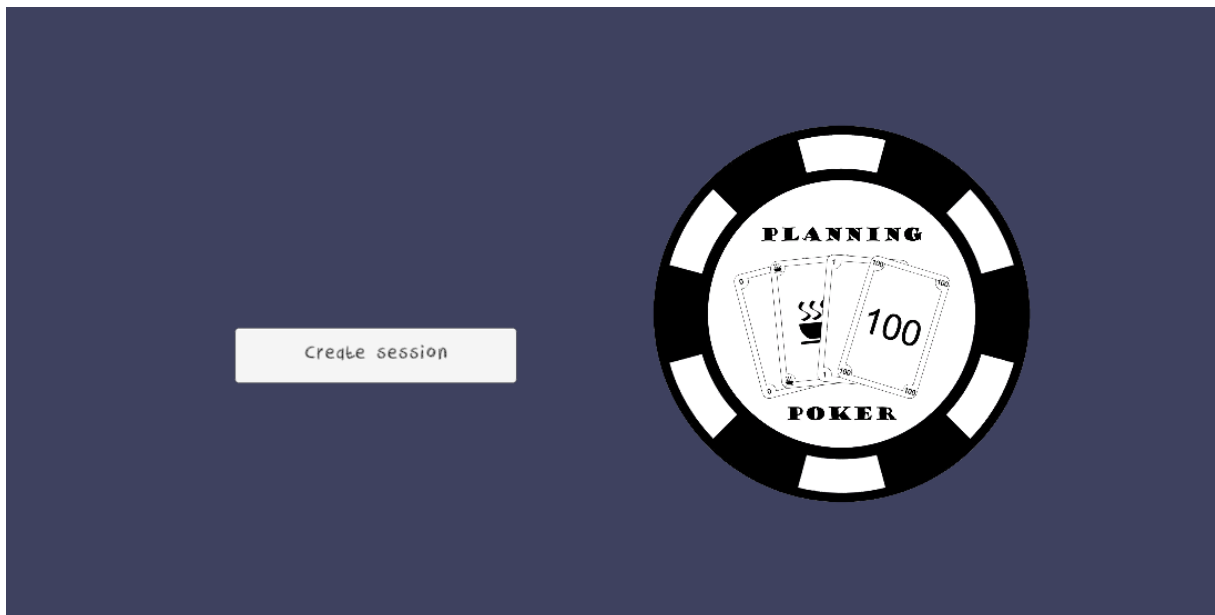
- Tests de création des classes USData et USJsonFile, nos classes de gestion des données, ainsi que la vérification de la sauvegarde et de la charge de fichiers JSON dans l'application.
- Test de la fonction ProcessAnswerByGameMode qui récupère l'ensemble des votes des participants et choisit en fonction du mode du jeu, soit la note finale à attribuer à la user story en cours, soit de refaire un vote sur cette même user story.
- Tests sur le calcul de la note en fonction des différents modes de vote : unanimité, médiane, moyenne, majorité absolue, majorité relative.
- Test de la fonction DisplayBoard qui affiche les users stories à estimer sur un tableau d'affichage pour tous les joueurs.

Enfin, un deuxième fichier doc.yaml gère la génération automatique de la documentation grâce à Doxygen. Les commentaires présents dans les summary de chaque fonctions et classes sont utilisés pour produire une documentation structurée. Cette documentation est automatiquement accessible sur GitHub Pages, mais avec un problème d'adresse. Il faut renseigner l'adresse <https://isweaz.github.io/ConceptionDeProjet/html/index.html>. La documentation est également accessible en téléchargeant le dossier html sur la branche gh pages.

4. Manuel utilisateur et fonctionnalités

Pour lancer le projet, il faut le cloner (git clone <https://github.com/iSweaz/ConceptionDeProjet.git> avec git bash), puis ouvrir la branche main qui contient un exécutable 'Build/ConceptionDeProjet.exe'. Pour le multijoueur, il faut donc lancer l'exécutable le nombre de fois qu'il y a de participants.

Le premier écran qui se lance est celui qui permet d'identifier s'il y a une session de planning poker qui est déjà existante ou non. Si aucune session n'existe, le bouton proposé est « Create session », qui permet d'accéder à la scène de création de réunion. Si une réunion existe déjà, le bouton affiché sera « Join session », qui permet de rejoindre la session en cours. Il ne peut donc y avoir qu'une seule session de planning poker ouverte en même temps dans le réseau LAN. Il faut que la session soit crée (c'est-à-dire que l'host soit dans le bureau de la réunion et non dans l'interface de création de session) pour que les autres utilisateurs puissent s'y connecter.



Menu de lancement de l'application

Du point de vue de l'hôte, pour créer une session, il faut entrer un pseudo, le nombre de participants à la réunion, la durée du chronomètre et le mode de vote choisi (cadre A). L'hôte peut également choisir d'importer un deck en sélectionnant dans sa machine un fichier JSON, ou bien d'en créer un directement depuis l'application (cadre B). Le deck choisi s'affiche dans l'interface de création de session (cadre C). Si l'hôte a sélectionné un deck qui comportait déjà des notes, il peut choisir de réévaluer les users stories déjà estimées ou non (cadre D). Une fois tous les champs remplis, le bouton de création de session s'active.

A

Pseudo

Moyenne

8

120

☒ Réévaluer toutes les users stories ?

D

C

Planning Poker

[00] Interface
Designer les différentes interfaces nécessaires

[04] JSON
Gestion (charge et sauvegarde) des fichiers JSON

[02] Connexion
Connexion d'un client à une session déjà existante

B

Create Deck

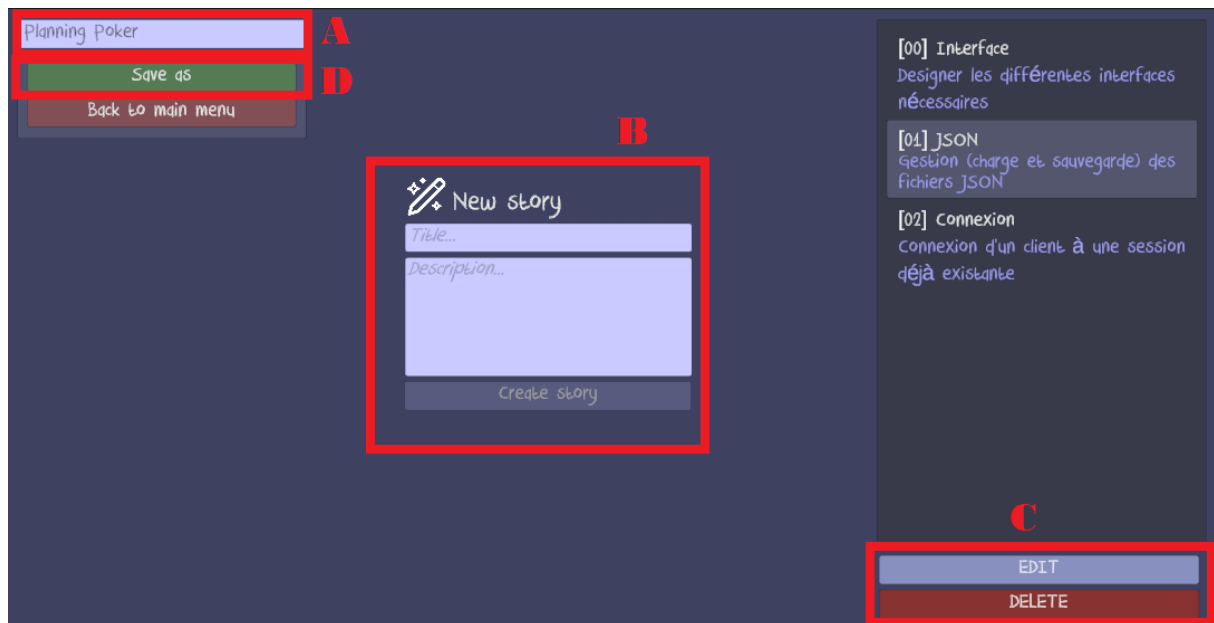
Modify deck

Import Deck

Create session

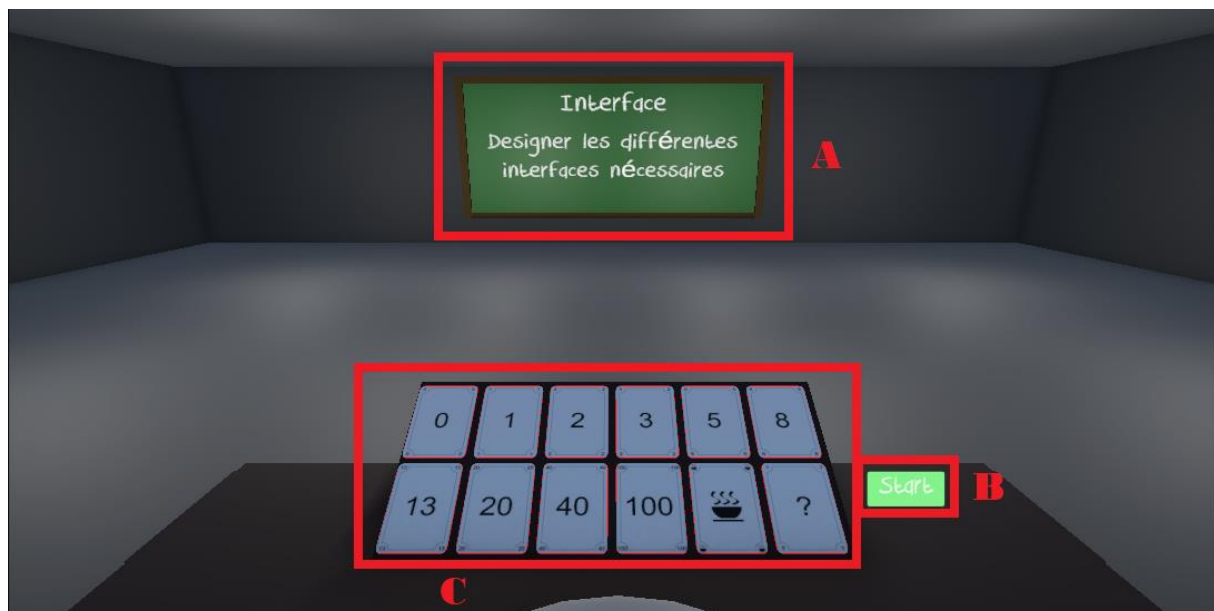
Interface de création de session

Pour créer ou modifier un deck depuis l'application, une nouvelle fenêtre s'affiche. Le titre du deck doit être renseigné en haut à gauche (cadre A). Le titre sera par défaut le nom du fichier JSON enregistré sur la machine. Pour créer une nouvelle user story, il faut renseigner son titre et la description, puis le bouton « Create story » s'active (cadre B). Les users stories déjà créées peuvent être modifiées ou supprimées en les sélectionnant et en cliquant sur les boutons correspondants (cadre C). Enfin, il faut sauvegarder le nouveau deck ou les changements faits en haut à gauche (cadre D).



Interface de création ou modification de deck

Une fois arrivé dans la réunion, la première user story à estimer s'affiche sur le tableau (cadre A). L'hôte peut choisir quand démarrer les votes grâce à un bouton « Start » (cadre B) qui n'est pas disponible pour les clients. Une fois le vote lancé, tout le monde peut voter en cliquant sur la carte désirée, qui s'entoure alors en vert (cadre C). Un chronomètre indiquant le temps restant pour voter s'affiche également (cadre D).



Interface de réunion pour l'hôte



Interface de réunion pour le client

Le fonctionnement du vote dépend du mode de vote choisi à l'origine par l'hôte. Pour le mode Unanimité, le score attribué est le score choisi par l'ensemble des participants, qui doivent se mettre d'accord. Le vote peut être recommencé autant de fois que nécessaire, avec un temps de pause pour permettre aux participants de discuter entre eux. Pour le mode Moyenne, le score attribué sera la moyenne de toutes les notes votées par les participants. Pour le mode Médiane, le score attribué sera la médiane de toutes les notes votées par les participants. Pour le mode Majorité relative, le score attribué est le score le plus présent parmi les notes votées par les participants. À la fin de chaque session de vote, le score s'affiche sur le tableau et l'hôte peut choisir de passer à la suivante, ou bien dans le cas de l'unanimité et que les participants ne se sont pas mis d'accord, de recommencer le vote. Le fichier JSON se sauvegarde automatiquement avec la note attribuée à chaque fin de vote.

En cas de vote pour la carte « café », l'hôte peut choisir d'accepter ou non la pause. Il ne peut y avoir qu'une seule pause par meeting. En cas de vote pour la carte « ? », l'hôte peut recommencer le vote.

Une fois que toutes les users stories du deck ont été traitées, un écran de fin s'affiche et précise que le fichier JSON a bien été sauvegardé. Les participants peuvent quitter l'application.